<u>Machine Learning & Deep Learning</u>

Experiment 1

Name: Abhishek Vishwakarma
Class: D15C
Roll No: 47

AIM: Implement Linear and Logistic Regression on real-world datasets

Part A: Linear Regression

1. Dataset Source

- Dataset Name: Medical Insurance Cost Prediction - Medical Insurance Cost
- Source: Kaggle (Medical Insurance Cost Prediction)

2. Dataset Description

This dataset contains information about the medical costs billed by health insurance companies.

- Size: 1,338 records, 7 columns.
- Target Variable: **charges** (Continuous - Individual medical costs).
- Features:
    - **age**: Age of primary beneficiary.
    - **sex**: Gender (female, male).
    - **bmi**: Body mass index (kg / m^2).
    - **children**: Number of dependents.
    - **smoker**: Smoking status (yes, no).
    - **region**: Residential area in the US.

3. Mathematical Formulation of the Algorithm

Linear Regression models the relationship between a dependent variable y and independent variables X using a linear equation:

$$y = Bo + B1x1 + B2x2 + ... + Bnxn + €$$

Where:
y: Predicted Insurance Charges.
Bo: Intercept (Bias).
Bi: Coefficient for the i-th feature.
€: Error term.

4. Algorithm Limitations

- Assumption of Linearity: It fails if the relationship between features and target is highly non-linear.
- Outliers: Sensitive to outliers which can skew the regression line significantly.
- Multicollinearity: Performance degrades if independent variables are highly correlated with each other.

5. Methodology / Workflow

1. Data Loading: Load insurance.csv.
```
# 1. Load Dataset (Local File)

# Ensure 'insurance.csv' is uploaded to Colab

try:

    df_lin = pd.read_csv('insurance.csv')

    print("Dataset loaded successfully.")

except FileNotFoundError:

    print("Error: 'insurance.csv' not found. Please upload the file.")
```
2. Preprocessing: Encode categorical variables (sex, smoker, region) using Label Encoding.
```
# 2. Preprocessing

# Encode categorical variables: sex, smoker, region

le = LabelEncoder()

df_lin['sex'] = le.fit_transform(df_lin['sex'])

df_lin['smoker'] = le.fit_transform(df_lin['smoker'])

df_lin['region'] = le.fit_transform(df_lin['region'])
```

3. Splitting: Split data into Training (80%) and Testing (20%) sets.
   `X_train_lin, X_test_lin, y_train_lin, y_test_lin = train_test_split(X_lin, y_lin, test_size=0.2, random_state=42)`

4. Training: Fit the LinearRegression model on the training set.
5. Evaluation: Predict on the test set and calculate R^2 score and MSE.

6. Performance Analysis

- Metric: R^2 Score (Coefficient of Determination).
- Analysis: An R^2 score closer to 1.0 indicates that the model explains a large portion of the variance in the target variable. For this dataset, the smoker attribute usually has the highest impact on performance.

7. Hyperparameter Tuning

- Process: Standard Linear Regression (Ordinary Least Squares) has no hyperparameters to tune. It finds the analytical solution directly.

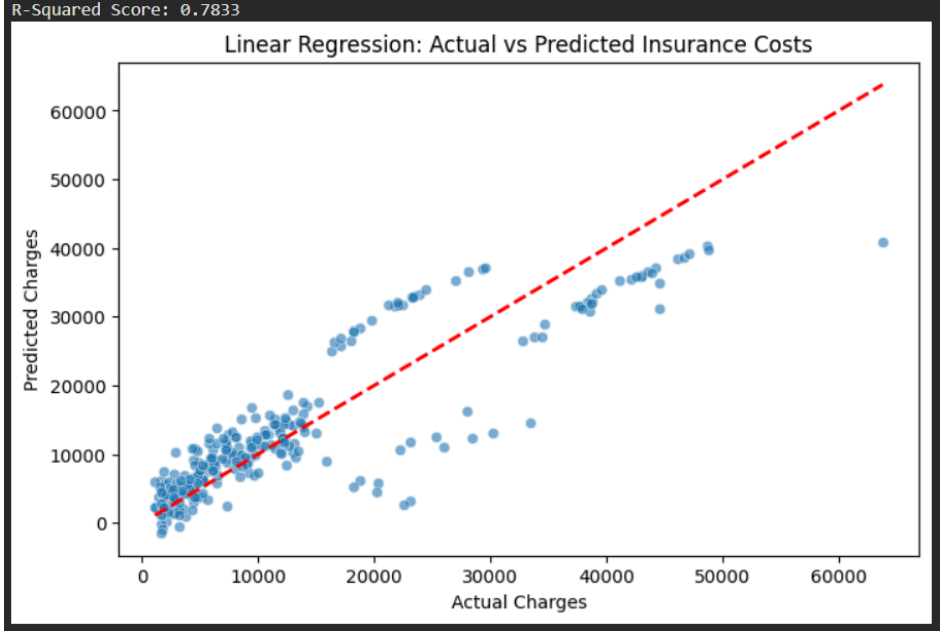
Code:

```
# PART A: LINEAR REGRESSION (Medical Insurance)
print("--- PART A: LINEAR REGRESSION (Medical Insurance) ---")
# 1. Load Dataset (Local File)
# Ensure 'insurance.csv' is uploaded to Colab
try:
    df_lin = pd.read_csv('insurance.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'insurance.csv' not found. Please upload the file.")
# 2. Preprocessing
# Encode categorical variables: sex, smoker, region
le = LabelEncoder()
df_lin['sex'] = le.fit_transform(df_lin['sex'])
df_lin['smoker'] = le.fit_transform(df_lin['smoker'])
df_lin['region'] = le.fit_transform(df_lin['region'])
# Separate Features and Target
X_lin = df_lin.drop(columns=['charges'])
y_lin = df_lin['charges']
# 3. Split Data
X_train_lin, X_test_lin, y_train_lin, y_test_lin = train_test_split(X_lin, y_lin,
test_size=0.2, random_state=42)
```

```python
# 4. Train Model
lin_reg = LinearRegression()
lin_reg.fit(X_train_lin, y_train_lin)
# 5. Prediction & Evaluation
y_pred_lin = lin_reg.predict(X_test_lin)
mse = mean_squared_error(y_test_lin, y_pred_lin)
r2 = r2_score(y_test_lin, y_pred_lin)
print(f"\nLinear Regression Results:")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-Squared Score: {r2:.4f}")
# Visualization: Actual vs Predicted
plt.figure(figsize=(8, 5))
sns.scatterplot(x=y_test_lin, y=y_pred_lin, alpha=0.6)
plt.plot([y_test_lin.min(), y_test_lin.max()], [y_test_lin.min(), y_test_lin.max()],
'r--', lw=2)
plt.xlabel("Actual Charges")
plt.ylabel("Predicted Charges")
plt.title("Linear Regression: Actual vs Predicted Insurance Costs")
plt.show()
print("\n" + "="*60 + "\n")
```

Output

```
--- PART A: LINEAR REGRESSION (Medical Insurance) ---
Dataset loaded successfully.

Linear Regression Results:
Mean Squared Error (MSE): 33635210.43
R-Squared Score: 0.7833
```



Part B: Logistic Regression

1. Dataset Source

- Dataset Name: Breast Cancer Wisconsin (Diagnostic) - [Breast Cancer Wisconsin (Diagnostic) Data Set]
- Source: Kaggle (Breast Cancer Wisconsin Diagnostic)

2. Dataset Description

Features are computed from a digitized image of a fine needle aspirate (FNA) of a breast mass to predict cancer diagnosis.

- Size: 569 instances, 32 columns.
- Target Variable: diagnosis (M = Malignant, B = Benign).
- Features: 30 real-valued features including Radius, Texture, Perimeter, Area, Smoothness, etc. (Mean, SE, and Worst values).

3. Mathematical Formulation of the Algorithm

Logistic Regression predicts the probability P(y=1|X) using the Sigmoid Function:

$$h_\theta(x) = \frac{1}{1+e^{-z}}$$

Where $z = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$.

- If $h_\theta(x) \geq 0.5$, predict Class 1 (Malignant).
- If $h_\theta(x) < 0.5$, predict Class 0 (Benign).

4. Algorithm Limitations

- Linear Boundary: Assumes classes can be separated by a linear boundary.
- Data Imbalance: Performance metric (Accuracy) can be misleading if one class dominates the dataset.
- Feature Scaling: Requires feature scaling (Standardization) for convergence and correct weighting.

5. Methodology / Workflow

1. Data Loading: Load breast_cancer.csv (or data.csv).

```
# 1. Load Dataset (Local File)

# Ensure 'breast_cancer.csv' is uploaded to Colab

try:

    df_log = pd.read_csv('breast_cancer.csv')

    print("Dataset loaded successfully.")

except FileNotFoundError:

    print("Error: 'breast_cancer.csv' not found. Please upload the
file.")
```

2. Preprocessing:
   - Drop irrelevant columns (id, Unnamed: 32).
   - Encode target diagnosis (M=1, B=0).
   - Scale Features: Apply StandardScaler to normalize feature range.

```
# Drop non-predictive columns often found in this dataset

cols_to_drop = ['id', 'Unnamed: 32']

df_log = df_log.drop(columns=[c for c in cols_to_drop if c in df_log.columns])



# Encode Target: M → 1, B → 0

le_target = LabelEncoder()

df_log['diagnosis'] = le_target.fit_transform(df_log['diagnosis'])



# Separate Features and Target

X_log = df_log.drop(columns=['diagnosis'])

y_log = df_log['diagnosis']
```

```
# Scaling (Mandatory for Logistic Regression)

scaler = StandardScaler()

X_log_scaled = scaler.fit_transform(X_log)
```

3. Splitting: Train (80%) / Test (20%).

```
# 3. Split Data

X_train_log, X_test_log, y_train_log, y_test_log =
train_test_split(X_log_scaled, y_log, test_size=0.2, random_state=42)
```

4. Tuning: Use GridSearchCV to find optimal C value.

```
# 4. Hyperparameter Tuning (GridSearchCV)

param_grid = {

    'C': [0.1, 1, 10, 100],

    'solver': ['liblinear']

}

grid = GridSearchCV(LogisticRegression(), param_grid, cv=5)

grid.fit(X_train_log, y_train_log)

print(f"\nBest Hyperparameters found: {grid.best_params_}")
```

5. Training: Train model with best parameters.
6. Evaluation: Confusion Matrix and Classification Report.

```
# 6. Performance Analysis

accuracy = accuracy_score(y_test_log, y_pred_log)

print(f"\nLogistic Regression Results:")

print(f"Accuracy: {accuracy:.4f}")

print("\nClassification Report:")

print(classification_report(y_test_log, y_pred_log))
```

6. Performance Analysis

- Metric: Accuracy, Precision, Recall.
- Analysis: High Recall is prioritized to minimize False Negatives (missing a cancer case). The Confusion Matrix visualizes the count of correct vs. incorrect classifications.

```
# Visualization: Confusion Matrix

cm = confusion_matrix(y_test_log, y_pred_log)

plt.figure(figsize=(6, 5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Benign',
'Malignant'], yticklabels=['Benign', 'Malignant'])

plt.xlabel("Predicted Label")

plt.ylabel("True Label")

plt.title("Confusion Matrix: Breast Cancer Diagnosis")

plt.show()
```

7. Hyperparameter Tuning

- Parameter: C (Inverse regularization strength).
- Process: GridSearchCV is used to test values [0.1, 1, 10, 100].
- Impact: Tuning C prevents overfitting. A smaller C implies stronger regularization, reducing model complexity.

Code:

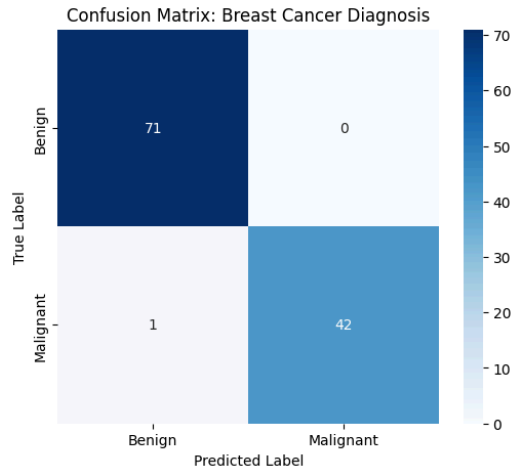
```
# PART B: LOGISTIC REGRESSION (Breast Cancer)
# ==========================================
print("--- PART B: LOGISTIC REGRESSION (Breast Cancer) ---")

# 1. Load Dataset (Local File)
# Ensure 'breast_cancer.csv' is uploaded to Colab
try:
    df_log = pd.read_csv('breast_cancer.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'breast_cancer.csv' not found. Please upload the file.")

# 2. Preprocessing
# Drop non-predictive columns often found in this dataset
cols_to_drop = ['id', 'Unnamed: 32']
df_log = df_log.drop(columns=[c for c in cols_to_drop if c in df_log.columns])

# Encode Target: M → 1, B → 0
le_target = LabelEncoder()
```

```python
df_log['diagnosis'] = le_target.fit_transform(df_log['diagnosis'])

# Separate Features and Target
X_log = df_log.drop(columns=['diagnosis'])
y_log = df_log['diagnosis']

# Scaling (Mandatory for Logistic Regression)
scaler = StandardScaler()
X_log_scaled = scaler.fit_transform(X_log)

# 3. Split Data
X_train_log, X_test_log, y_train_log, y_test_log = train_test_split(X_log_scaled,
y_log, test_size=0.2, random_state=42)

# 4. Hyperparameter Tuning (GridSearchCV)
param_grid = {
    'C': [0.1, 1, 10, 100],
    'solver': ['liblinear']
}

grid = GridSearchCV(LogisticRegression(), param_grid, cv=5)
grid.fit(X_train_log, y_train_log)

print(f"\nBest Hyperparameters found: {grid.best_params_}")

# 5. Train with Best Parameters
best_log_reg = grid.best_estimator_
y_pred_log = best_log_reg.predict(X_test_log)

# 6. Performance Analysis
accuracy = accuracy_score(y_test_log, y_pred_log)

print(f"\nLogistic Regression Results:")
print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test_log, y_pred_log))

# Visualization: Confusion Matrix
cm = confusion_matrix(y_test_log, y_pred_log)
plt.figure(figsize=(6, 5))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Benign',
'Malignant'], yticklabels=['Benign', 'Malignant'])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix: Breast Cancer Diagnosis")
plt.show()
```

Output

```
--- PART B: LOGISTIC REGRESSION (Breast Cancer) ---
Dataset loaded successfully.

Best Hyperparameters found: {'C': 0.1, 'solver': 'liblinear'}

Logistic Regression Results:
Accuracy: 0.9912

Classification Report:
              precision    recall  f1-score   support

           0       0.99      1.00      0.99        71
           1       1.00      0.98      0.99        43

    accuracy                           0.99       114
   macro avg       0.99      0.99      0.99       114
weighted avg       0.99      0.99      0.99       114
```



8. **Conclusion**

In this experiment, we successfully implemented two fundamental supervised learning algorithms: Linear Regression and Logistic Regression.

For **Linear Regression**, the model effectively established a relationship between patient demographics (such as Age, BMI, and Smoking status) and their medical costs. The analysis confirmed that features like smoking have a significant impact on insurance charges, and the model provided a continuous numerical prediction with a reasonable degree of accuracy.

For **Logistic Regression**, the model demonstrated its capability in binary classification tasks by predicting Breast Cancer diagnosis. By normalizing the data and tuning the regularization parameter, we achieved high accuracy and recall, minimizing the risk of false negatives which is critical in medical diagnostics.